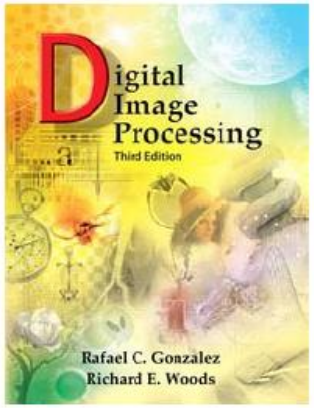
CSE 483: Computer Vision

Hough Transform

Dr. Mohamed Rehan

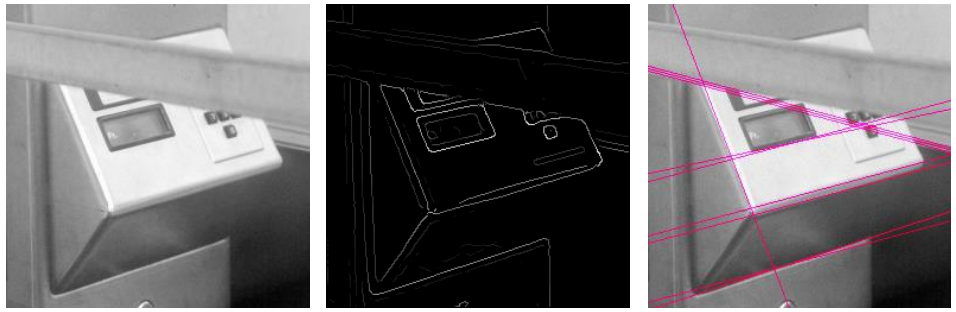
Text Book

- Digital Image Processing, Third edition, Rafael C. Gonzalez and Richard E. Woods, Pearson Education, Inc. 2008.
- Section: 10.2.7



Fitting

- Want to associate a model with observed features

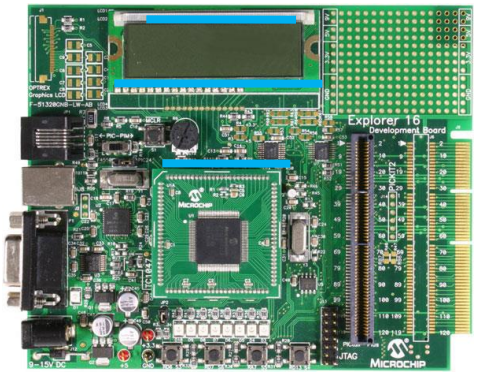


[Fig from Marszalek & Schmid, 2007]

For example, the model could be a line, a circle, or an arbitrary shape.

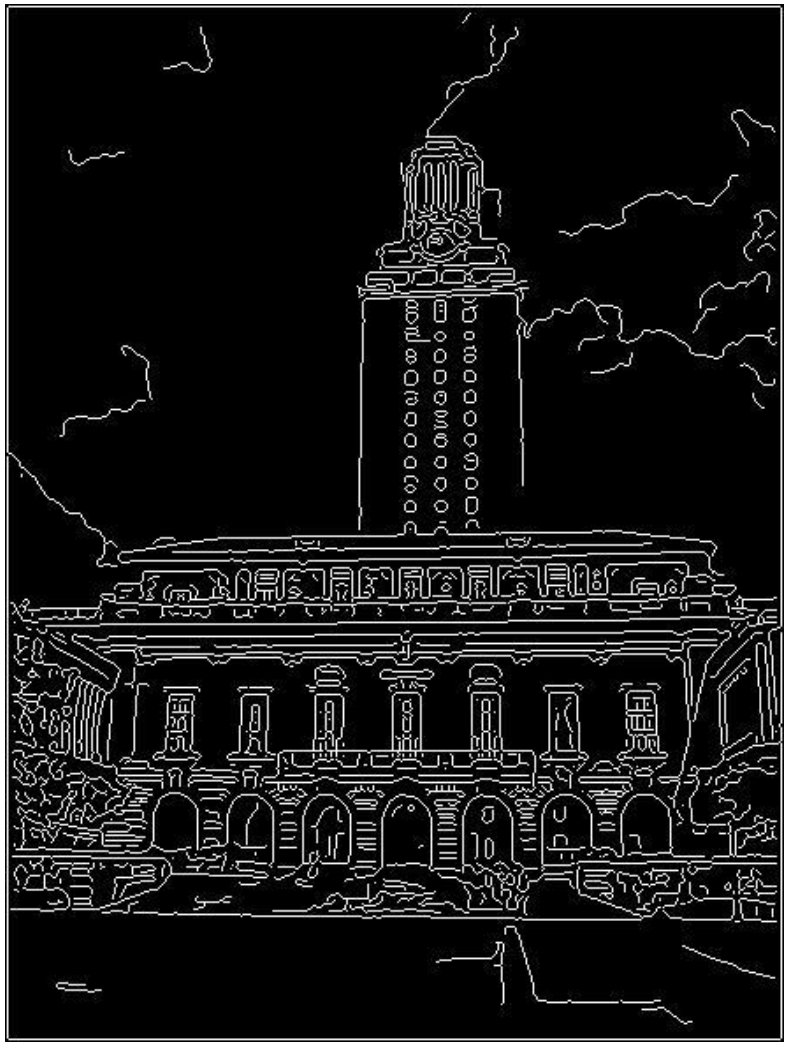
Example: Line fitting

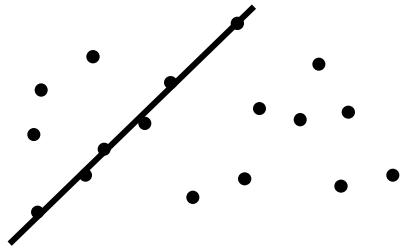
- Why fit lines?
  Many objects characterized by presence of straight lines



- Wait, why aren't we done just by running edge detection?

Difficulty of line fitting





- Extra edge points (clutter), multiple models:
  - which points go with which line, if any?
- Only some parts of each line detected, and some parts are missing:
  - how to find a line that bridges missing evidence?
- Noise in measured edge points, orientations:
  - how to detect true underlying parameters?

Voting

- It's not feasible to check all combinations of features by fitting a model to each possible subset.
- **Voting** is a general technique where we let the features *vote for all models that are compatible with it*.
  - Cycle through features, cast votes for model parameters.
  - Look for model parameters that receive a lot of votes.
- Noise & clutter features will cast votes too, *but* typically their votes should be inconsistent with the majority of "good" features.

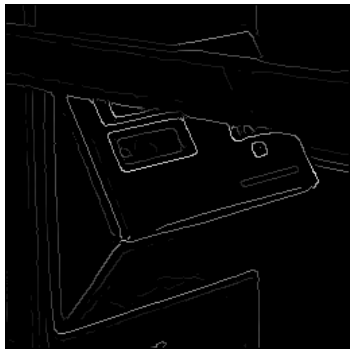
Fitting lines: Hough transform

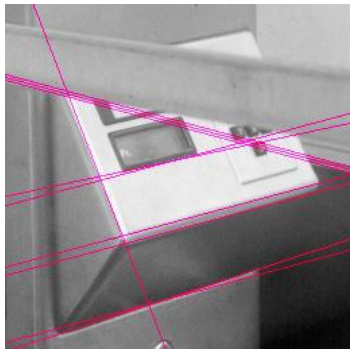
- Given points that belong to a line, what is the line?
- How many lines are there?
- Which points belong to which lines?

- **Hough Transform** is a voting technique that can be used to answer all of these questions.

  Main idea:

  1. Record vote for each possible line on which each edge point lies.
  2. Look for lines that get many votes.

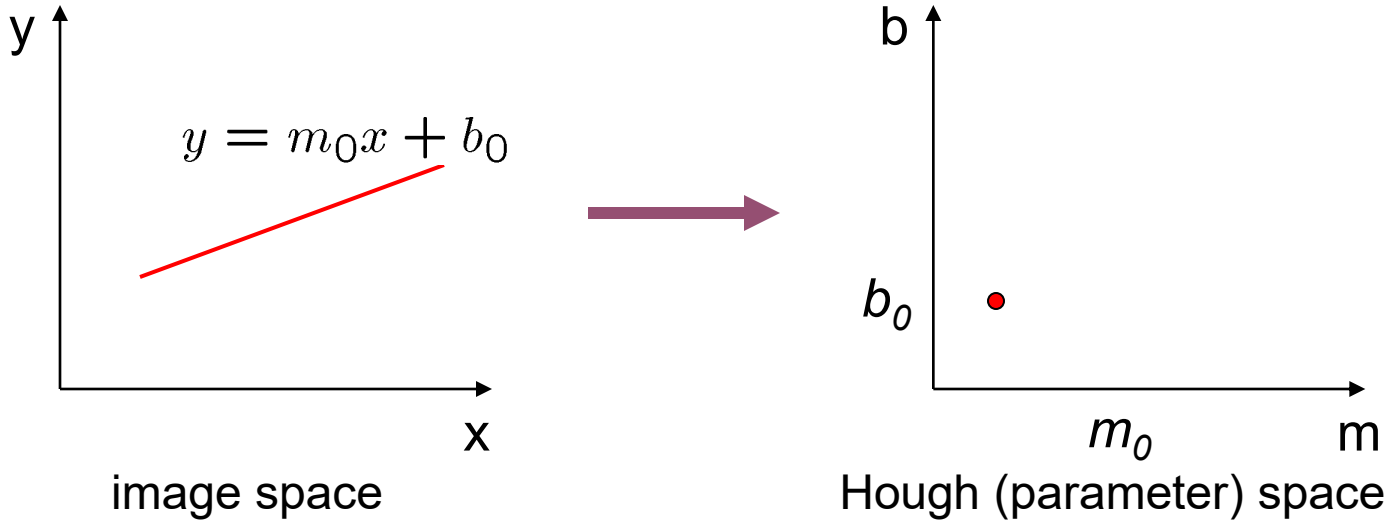




Hough transform is a feature extraction method for detecting simple shapes such as circles, lines etc. in an image

- Hough Transform takes the images created be edge detection operators but most of the time, edge map is disconnected
- Therefore Hough Transform is used to connect the disjoined edge points

Finding lines in an image: Hough space

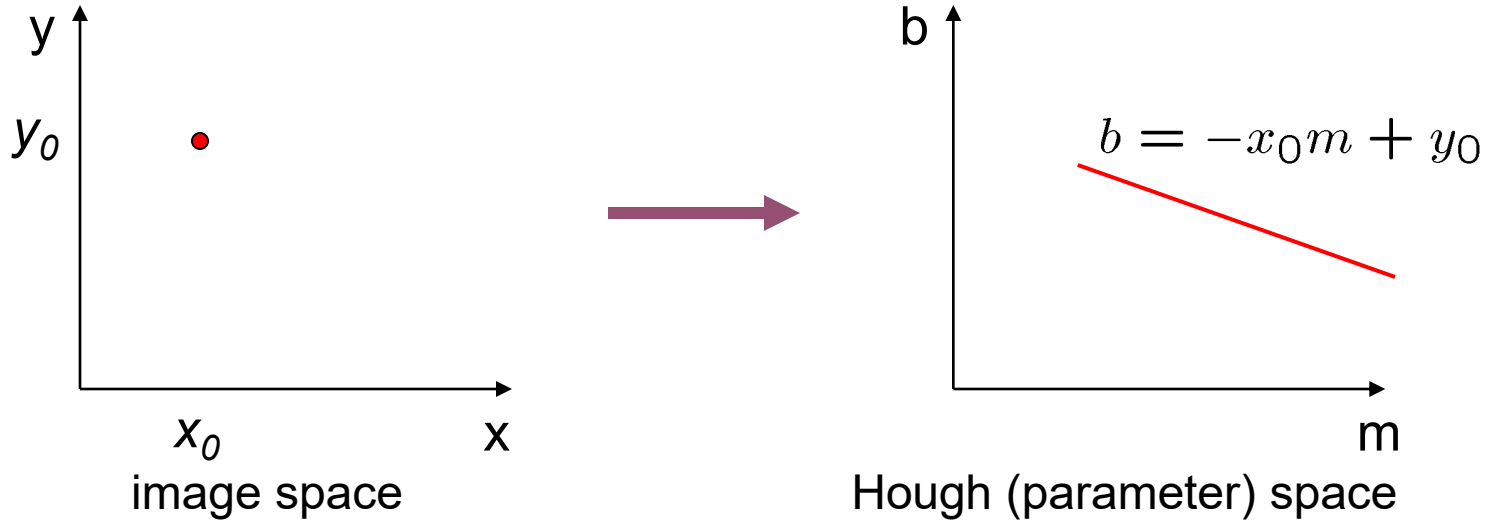


Connection between image (x,y) and Hough (m,b) spaces

- A line in the image corresponds to a point in Hough space
- To go from image space to Hough space:
  - given a set of points (x,y), find all (m,b) such that y = mx + b

Slide credit: Steve Seitz

Finding lines in an image: Hough space

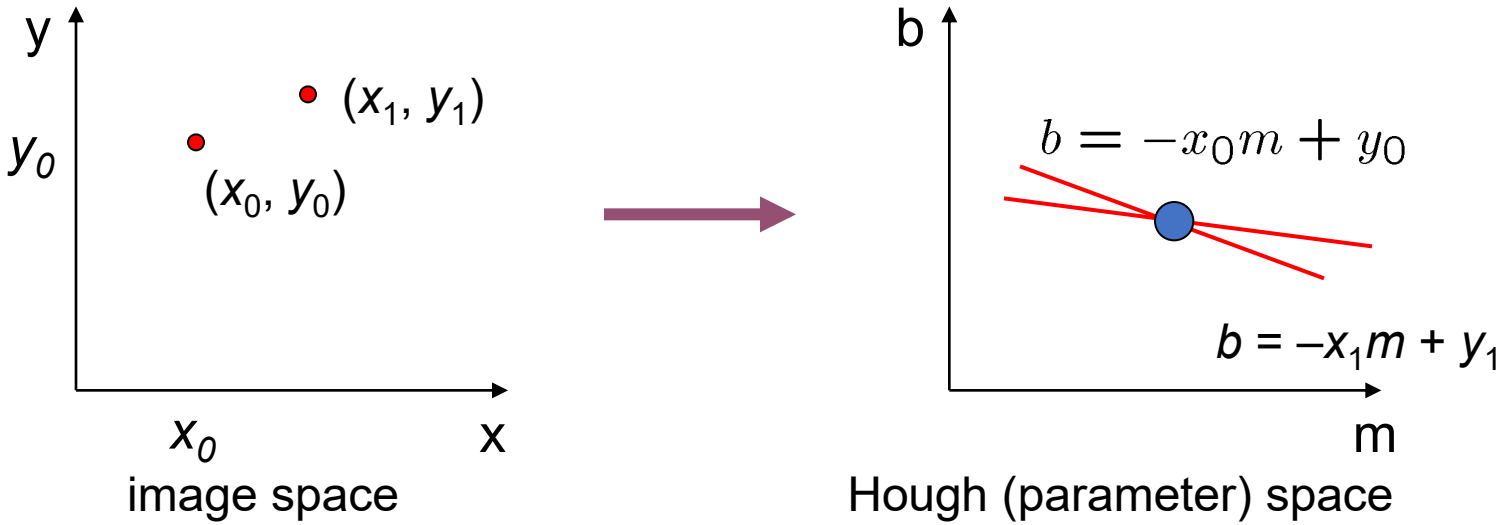


Connection between image (x,y) and Hough (m,b) spaces

- A line in the image corresponds to a point in Hough space
- To go from image space to Hough space:
  - given a set of points (x,y), find all (m,b) such that y = mx + b
- What does a point (x_0, y_0) in the image space map to?
  - Answer: the solutions of $b = -x_0 m + y_0$
  - this is a line in Hough space

Slide credit: Steve Seitz

Finding lines in an image: Hough space



What are the line parameters for the line that contains both (x_0, y_0) and (x_1, y_1)?

- It is the intersection of the lines $b = -x_0m + y_0$ and $b = -x_1m + y_1$

Finding lines in an image: Hough algorithm

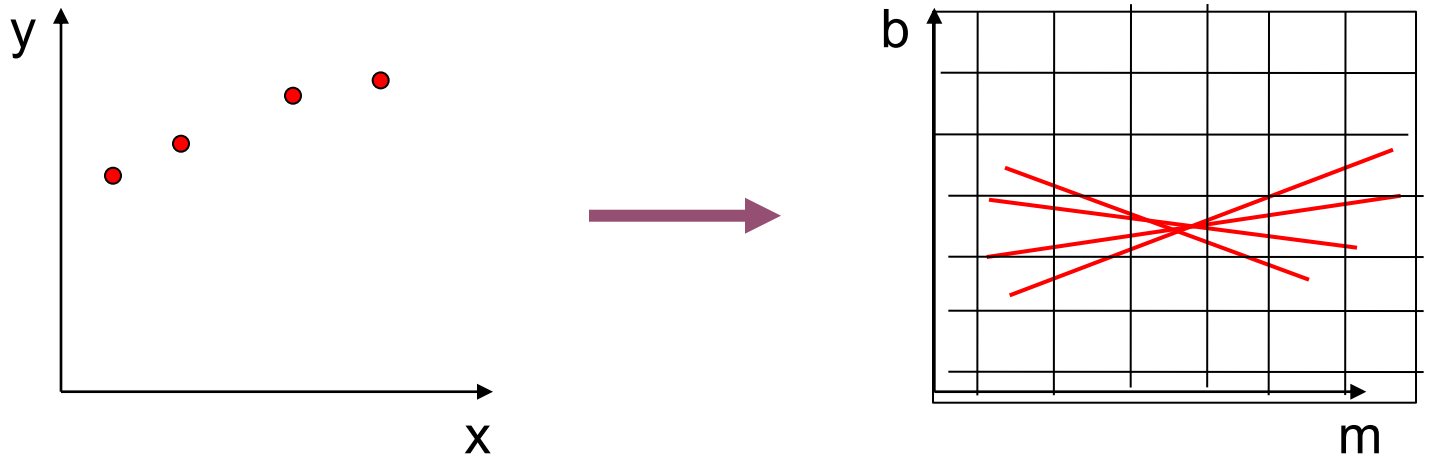


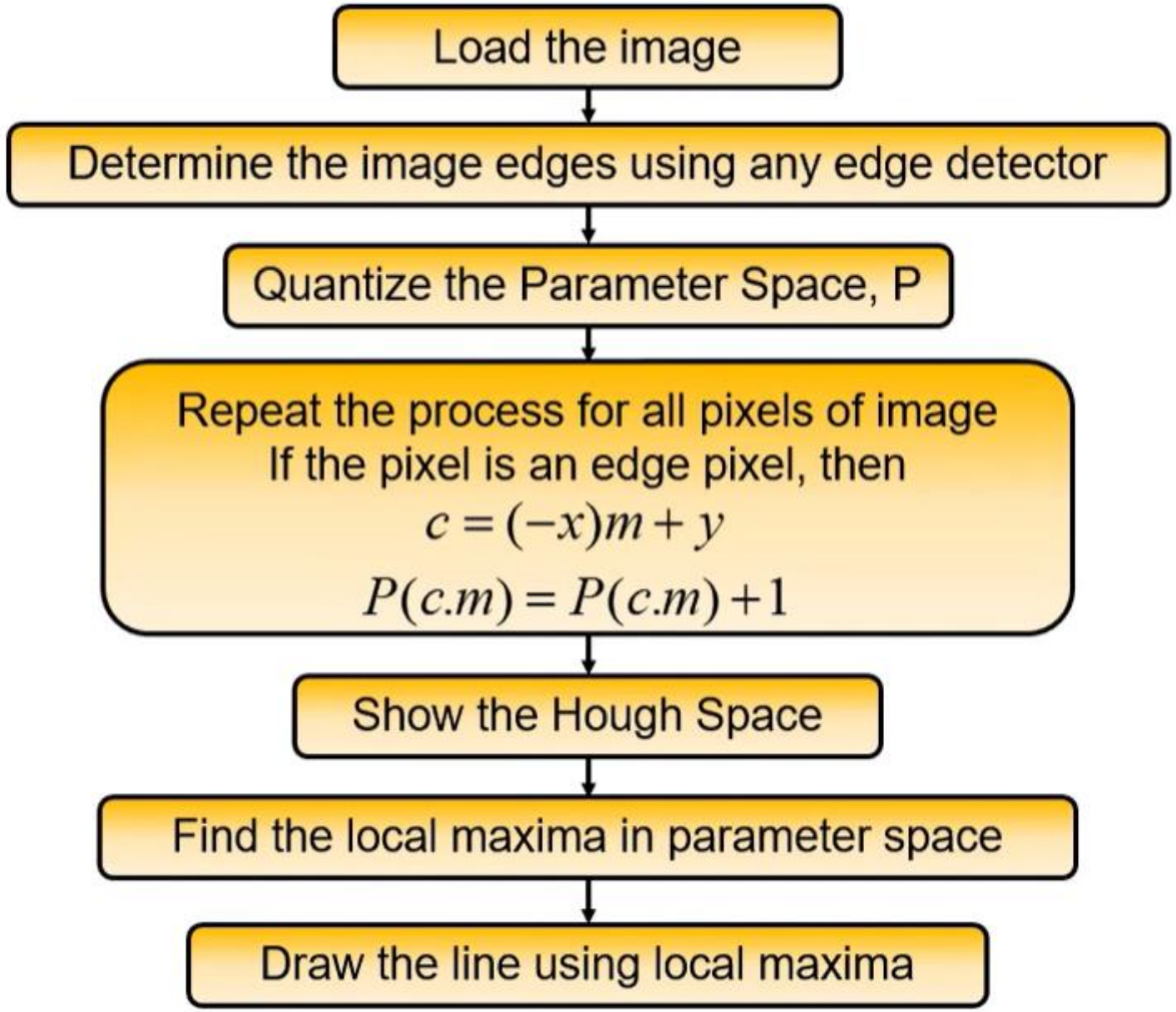
image space

Hough (parameter) space

How can we use this to find the most likely parameters (m,b) for the most prominent line in the image space?

- Let each edge point in image space *vote* for a set of possible parameters in Hough space
- Accumulate votes in discrete set of bins; parameters with the most votes indicate line in image space.

Algorithm

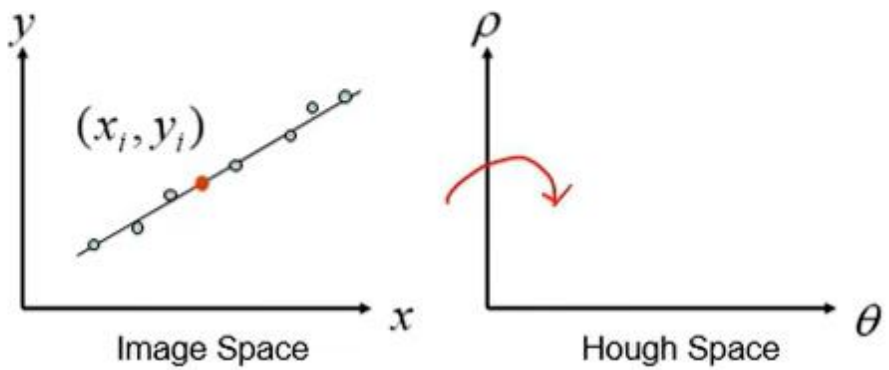

Load the image
Determine the image edges using any edge detector
Quantize the Parameter Space, P
Repeat the process for all pixels of image
If the pixel is an edge pixel, then
$c = (-x)m + y$
$P(c.m) = P(c.m) + 1$
Show the Hough Space
Find the local maxima in parameter space
Draw the line using local maxima

- Limitation of **previous algorithm** is that it does not work for vertical lines because they have infinity slope
- Therefore this **line** must be converted into **polar coordinates**

Equation of Line in polar:

$$\rho = x\cos\theta + y\sin\theta$$

Where,
Θ = angle between the line
ϱ = diameter



Load the image

Determine the image edges using any edge detector

Quantize the Parameter Space, P

Repeat the process for all pixels of image
If the pixel is an edge pixel, then for all Θ
- Calculate ϱ for pixel (x,y) and Θ
- Increment the position (ϱ, Θ) in accumulator array

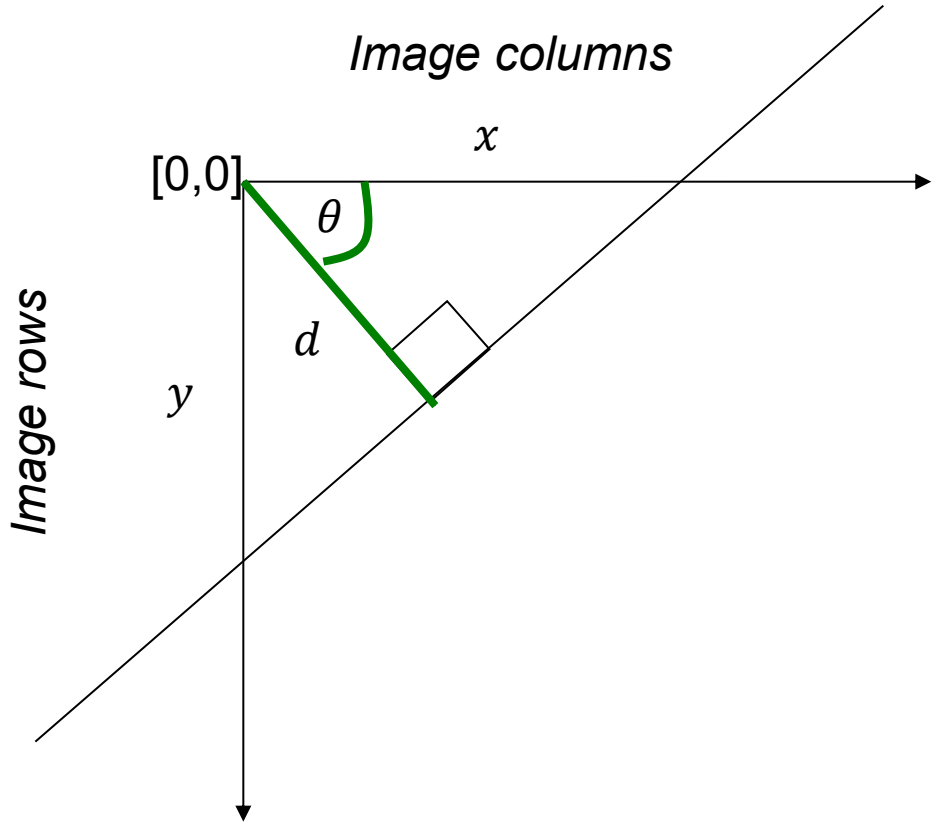
Show the Hough Space

Find the local maxima in parameter space

Draw the line using local maxima

Polar representation for lines

Issues with usual (*m,b*) parameter space: can take on infinite values, undefined for vertical lines.



d : perpendicular distance from line to origin

θ : angle the perpendicular makes with the x-axis

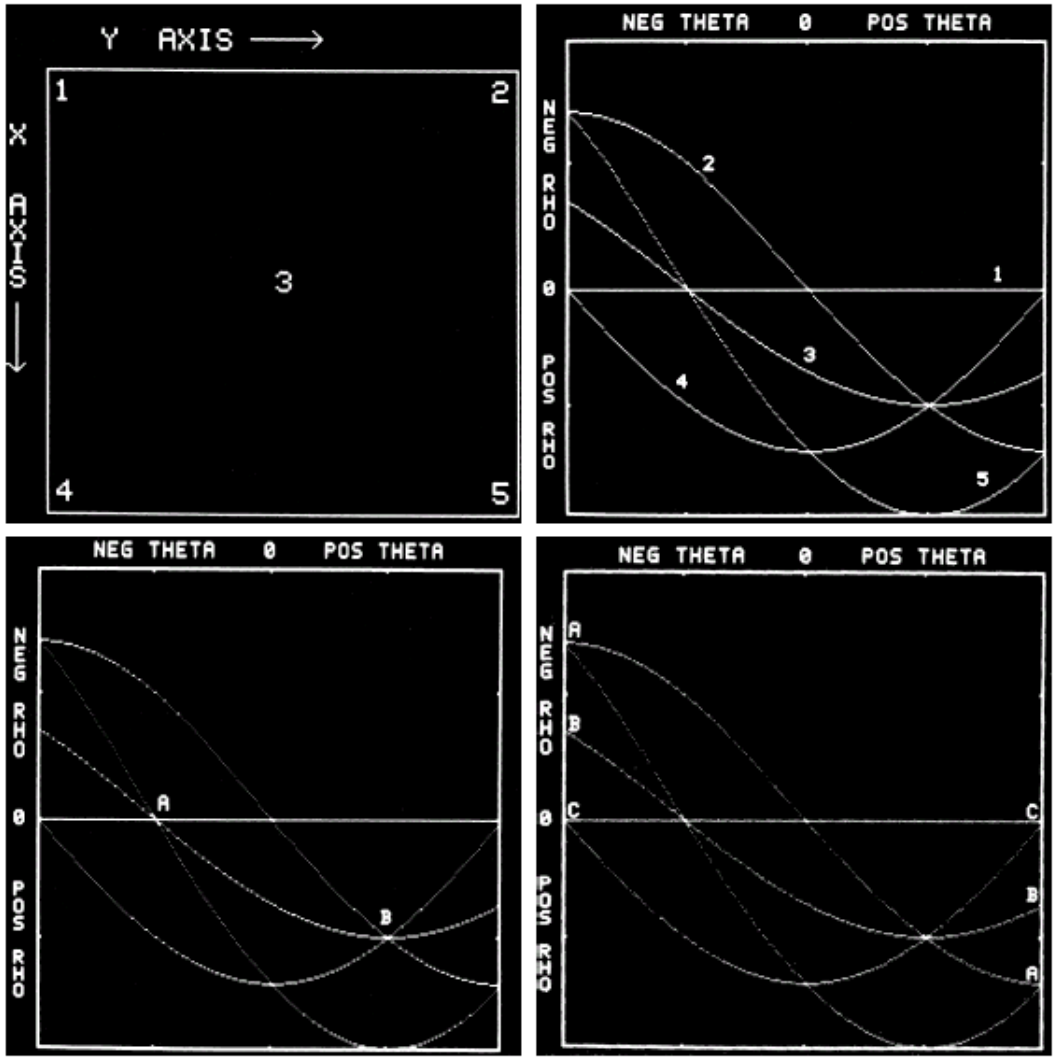
$$x\cos\theta - y\sin\theta = d$$

Point in image space → sinusoid segment in Hough space

Polar representation for lines

a b
c d

FIGURE 10.20
Illustration of the Hough transform. (Courtesy of Mr. D. R. Cate, Texas Instruments, Inc.)

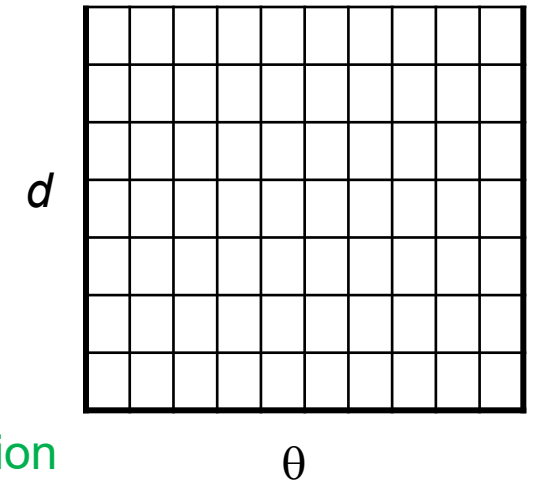


Hough transform algorithm

Using the polar parameterization:

$$x \cos\theta - y \sin\theta = d$$

H: accumulator array (votes)



Basic Hough transform algorithm

1. Initialize H[d, θ]=0
2. for each edge point I[x,y] in the image
   for θ = [θ_{min} to θ_{max}] // some quantization
   $d = x \cos\theta - y \sin\theta$
   H[d, θ] += 1
3. Find the value(s) of (d, θ) where H[d, θ] is maximum
4. The detected line in the image is given by

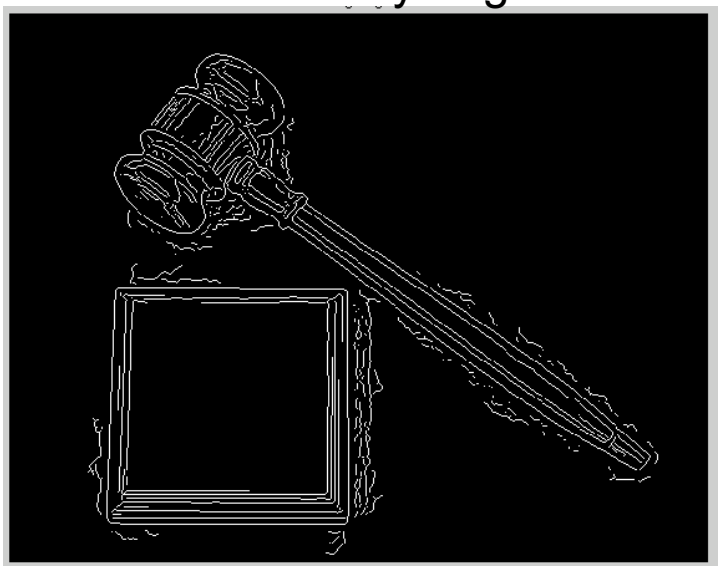
$$d = x \cos\theta - y \sin\theta$$

Time complexity (in terms of number of votes per pt)?

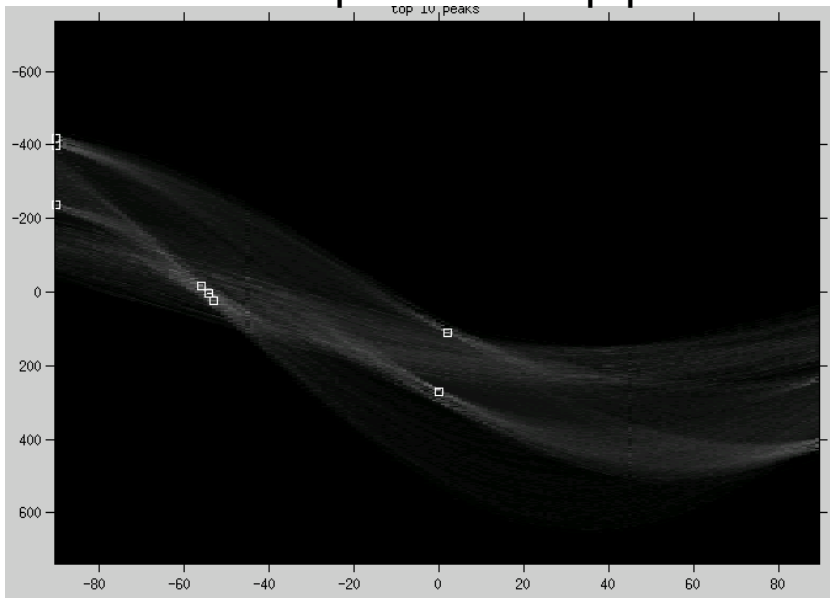
Source: Steve Seitz

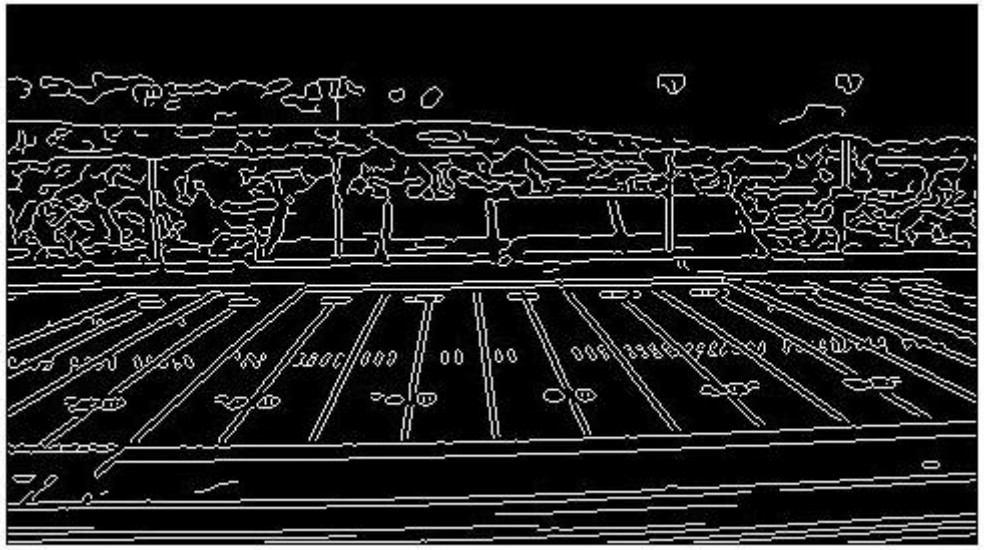
Original image

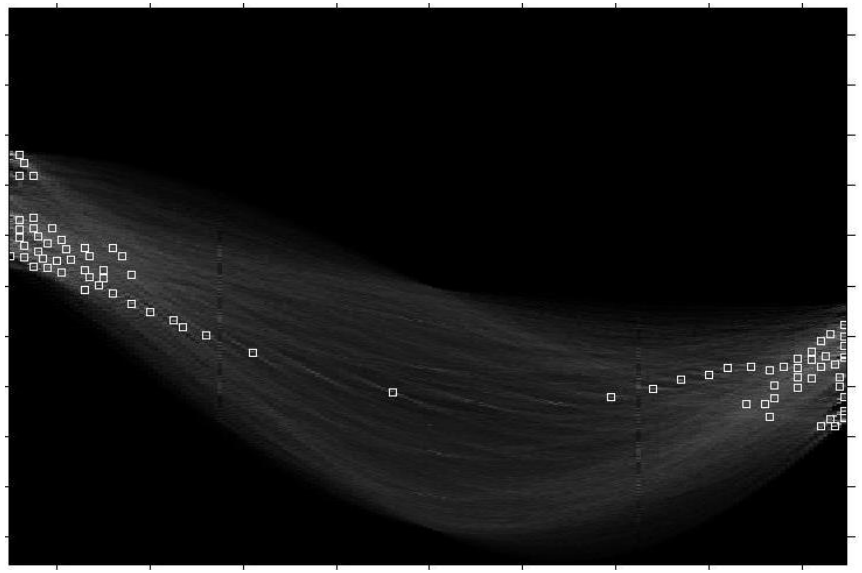
Canny edges

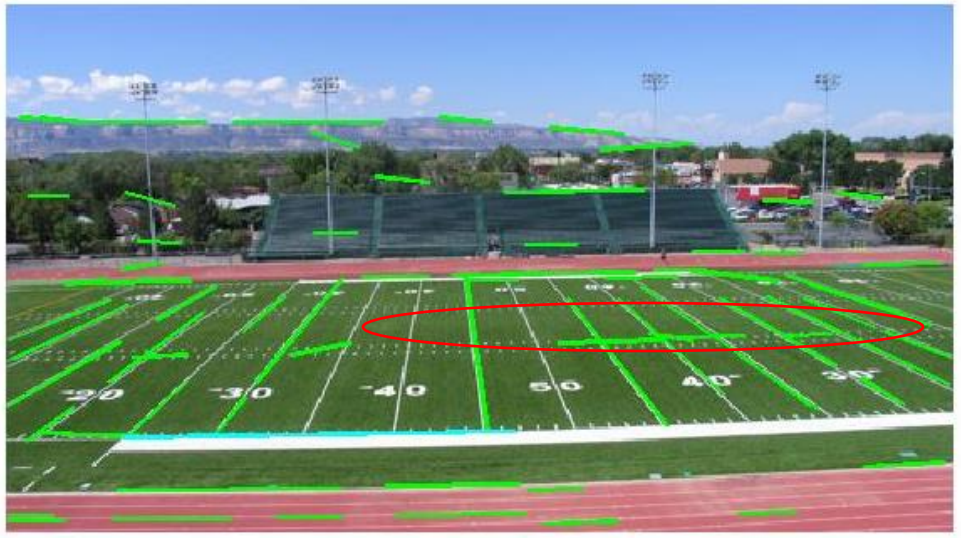


Vote space and top peaks



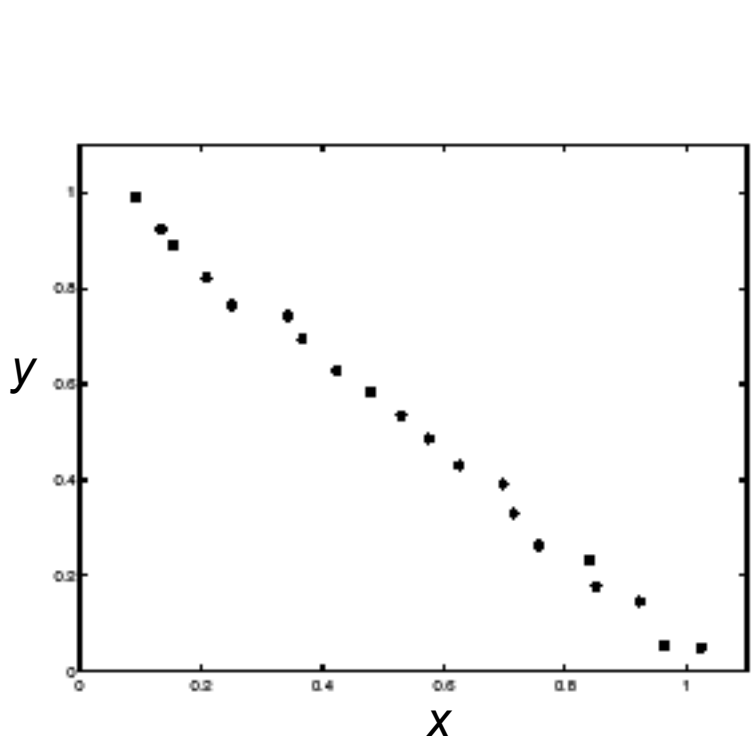






Showing longest segments found

Impact of noise on Hough



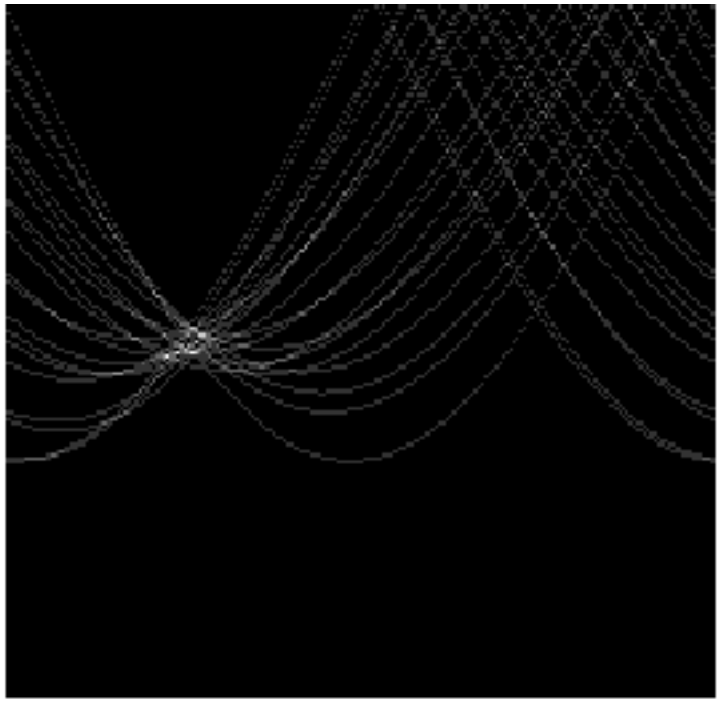


Image edge coordinates space

Votes

What difficulty does this present for an implementation?

Impact of noise on Hough

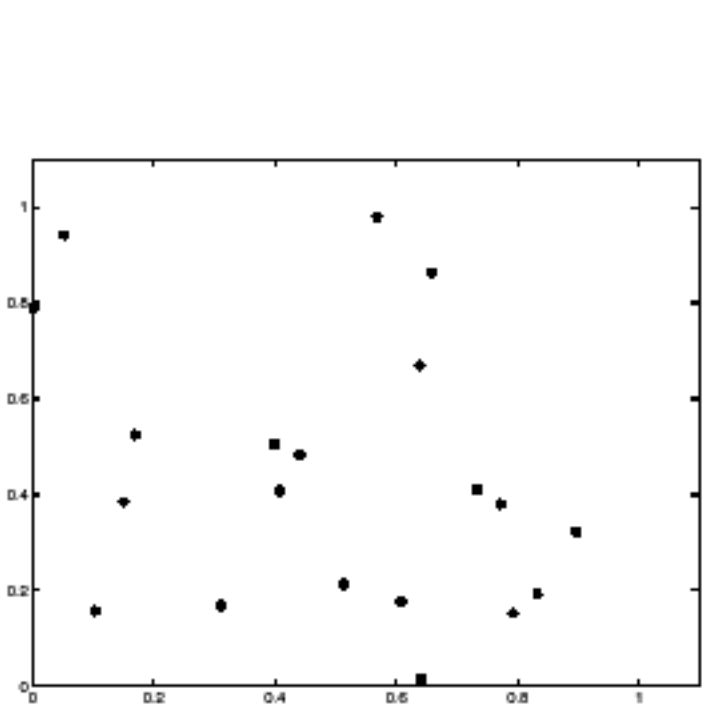
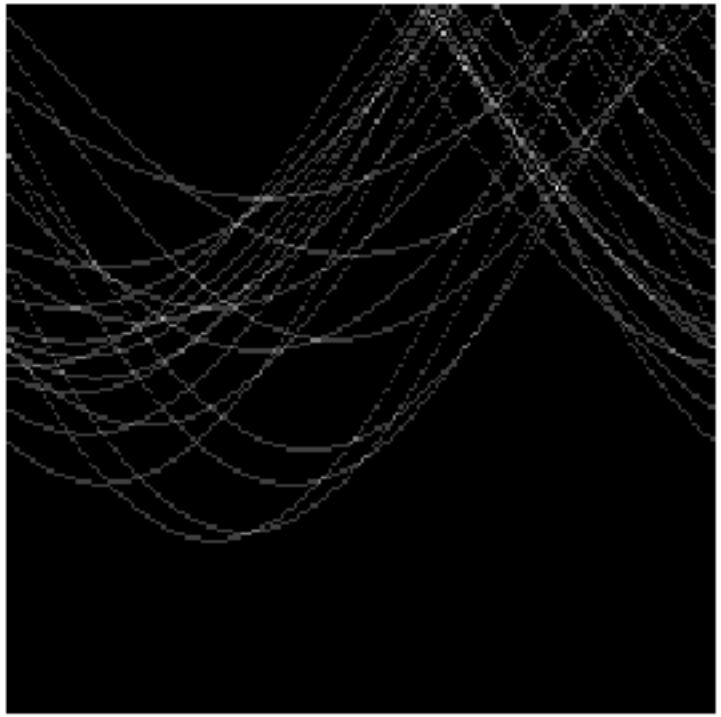


Image space
edge coordinates



Votes

Here, everything appears to be “noise”, or random edge points, but we still see peaks in the vote space.

Extensions

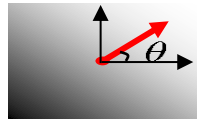
Extension 1: Use the image gradient

1. same
2. for each edge point I[x,y] in the image
   θ = gradient at (x,y)

$$d = x\cos\theta - y\sin\theta$$

   H[d, θ] += 1
3. same
4. same

(Reduces degrees of freedom)



$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

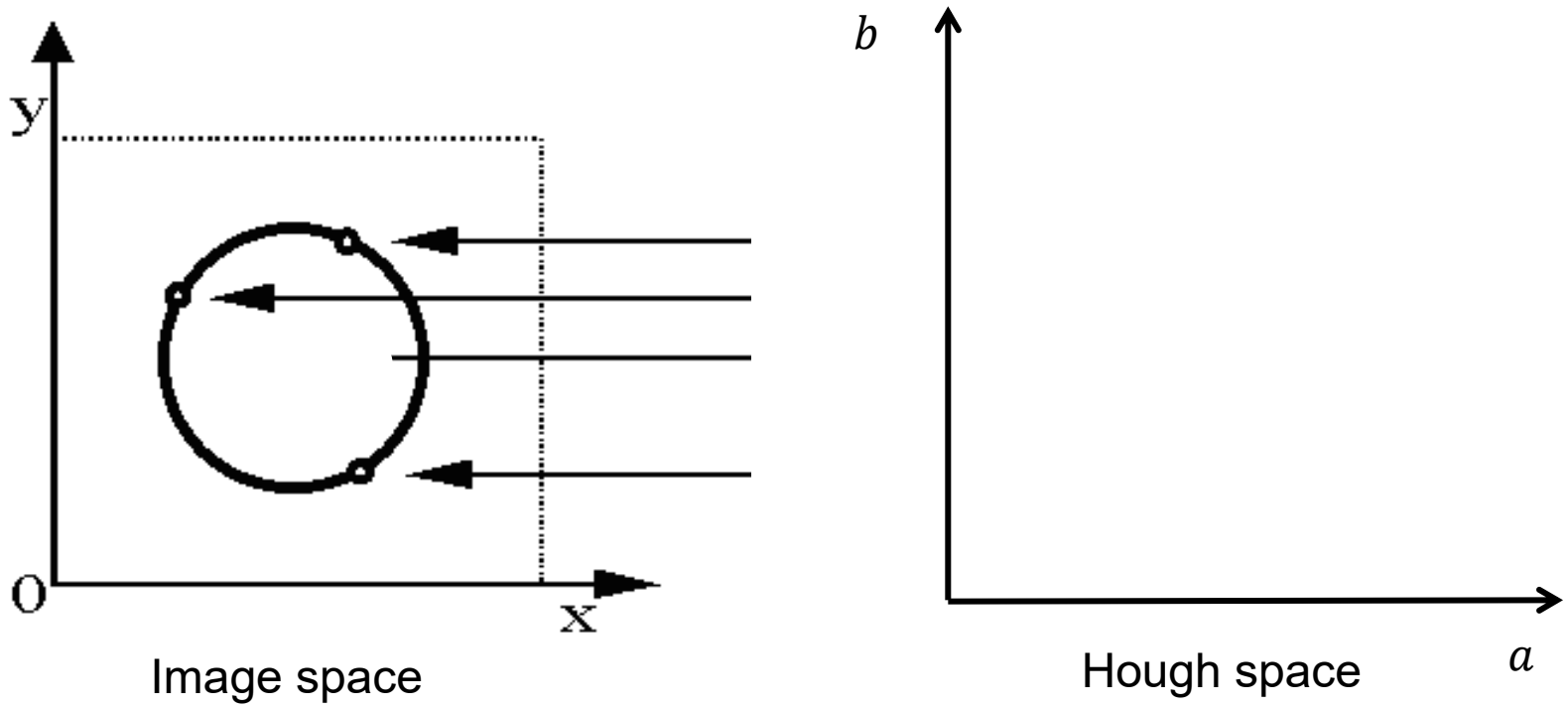
$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For a fixed radius *r*, unknown gradient direction

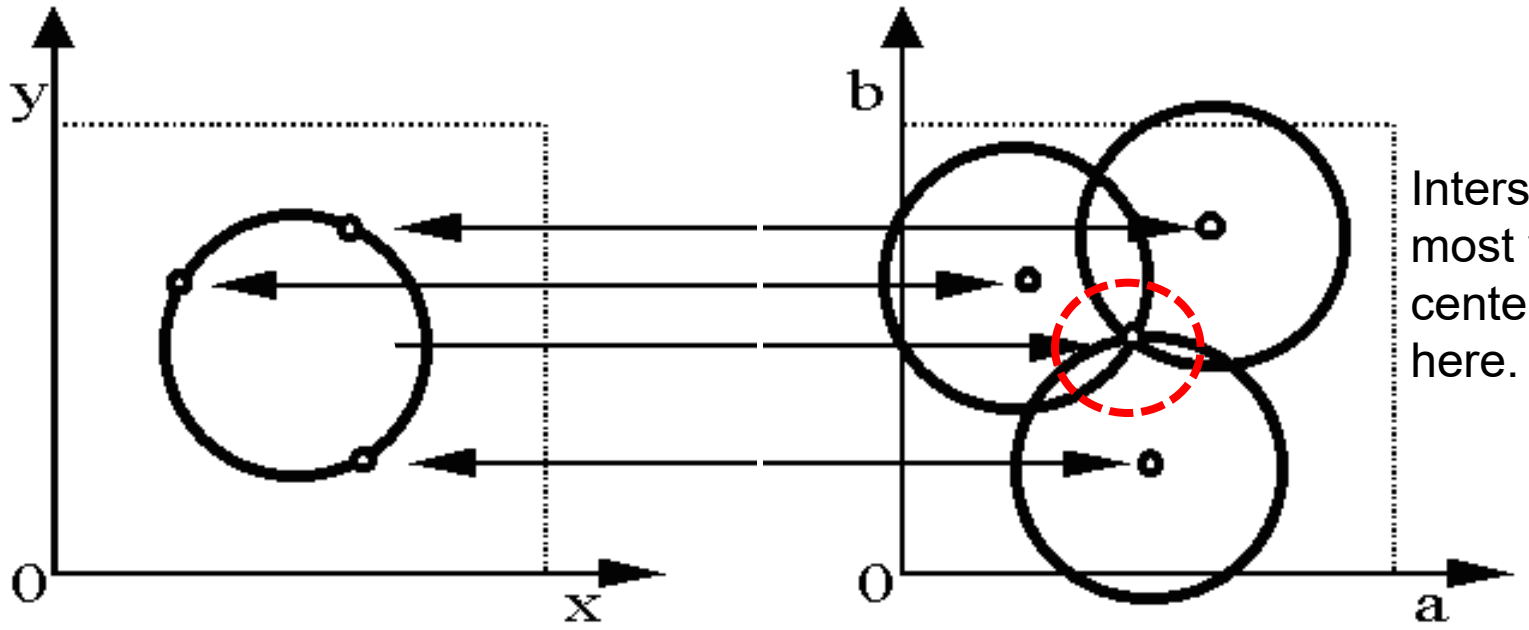


Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For a fixed radius r, unknown gradient direction



Hough transform for circles

- Circle: center (a,b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For an unknown radius r, unknown gradient direction

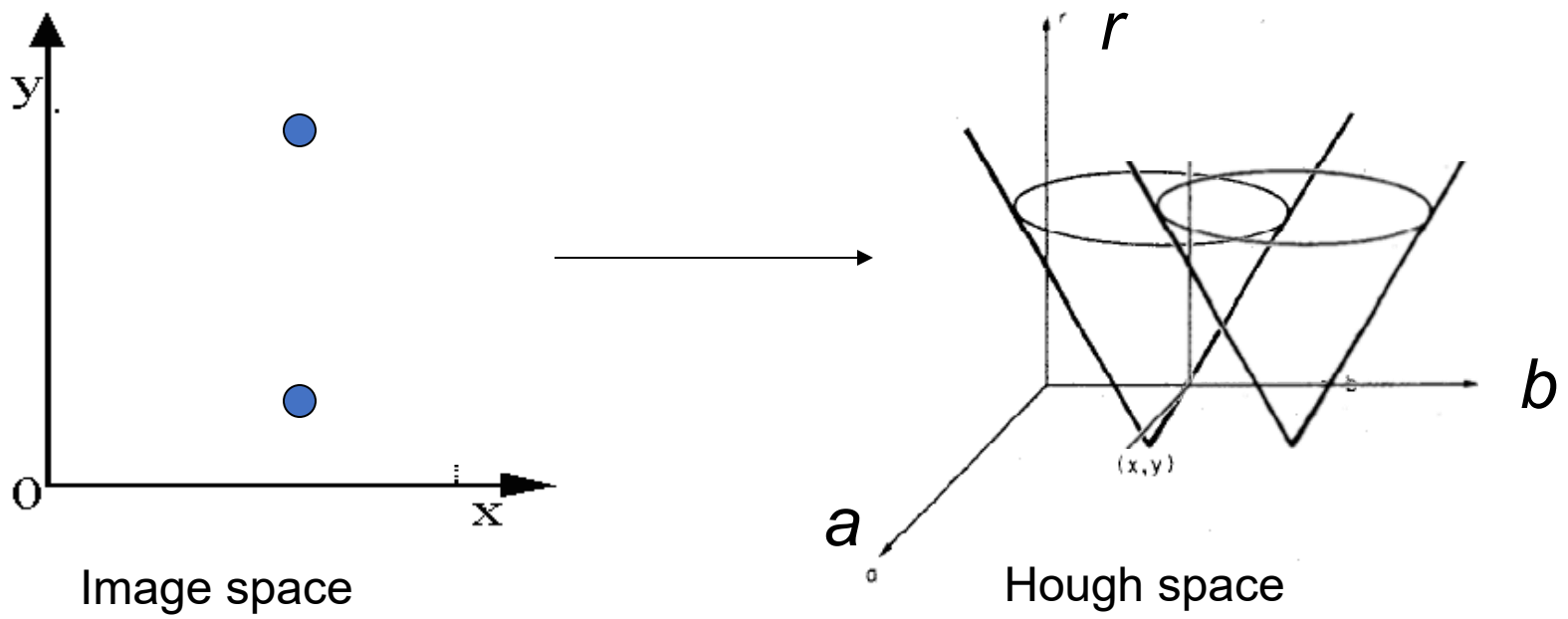
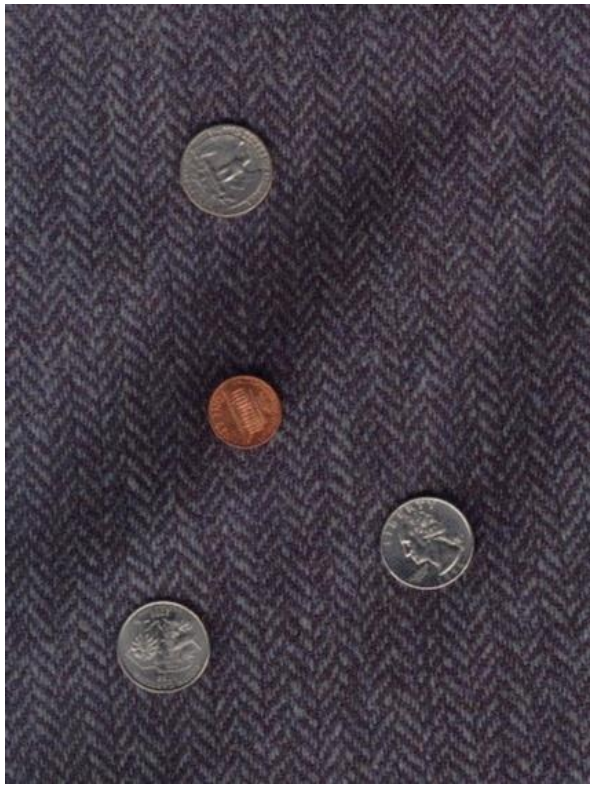


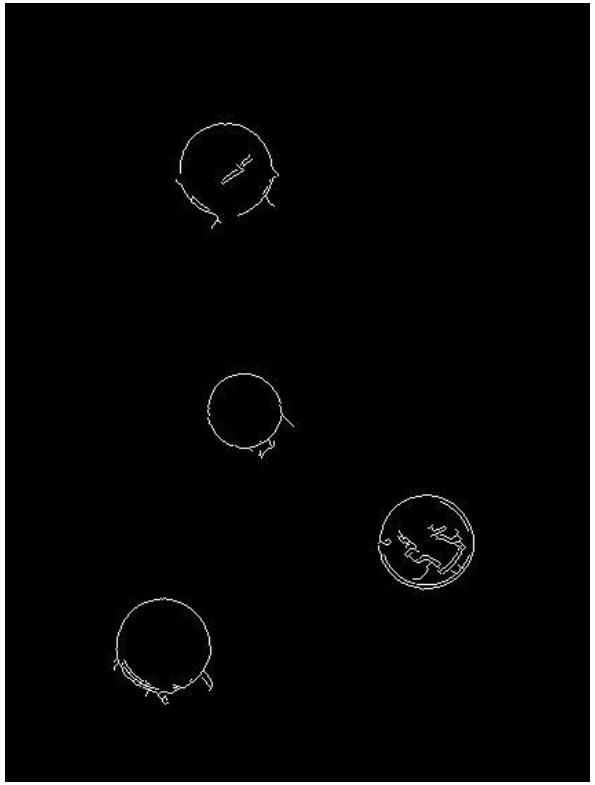
Image space Hough space

Example: detecting circles with Hough

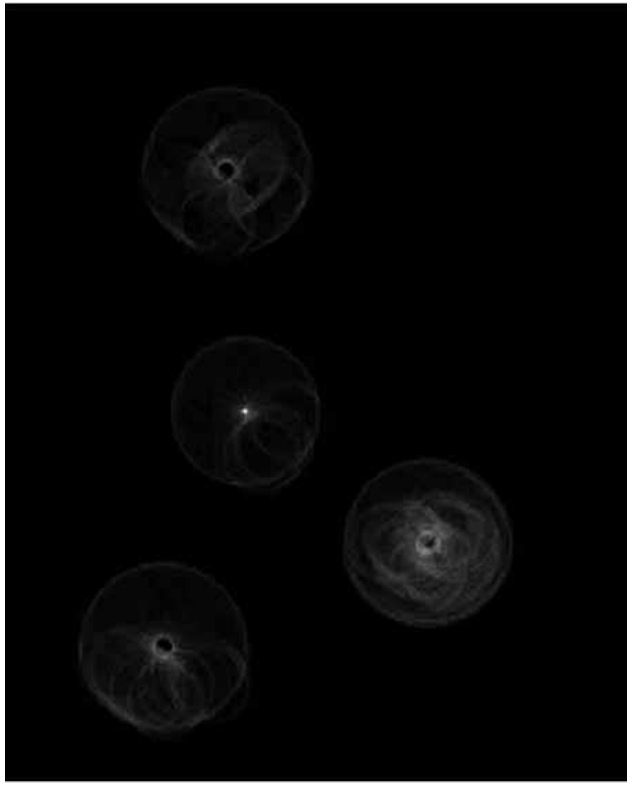
Original



Edges



Votes: Penny

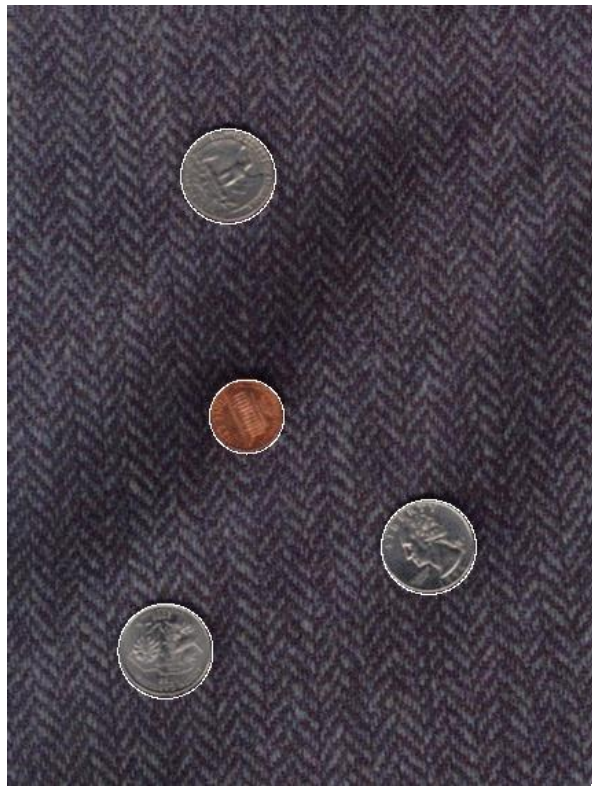


Note: a different Hough transform (with separate accumulators) was used for each circle radius (quarters vs. penny).

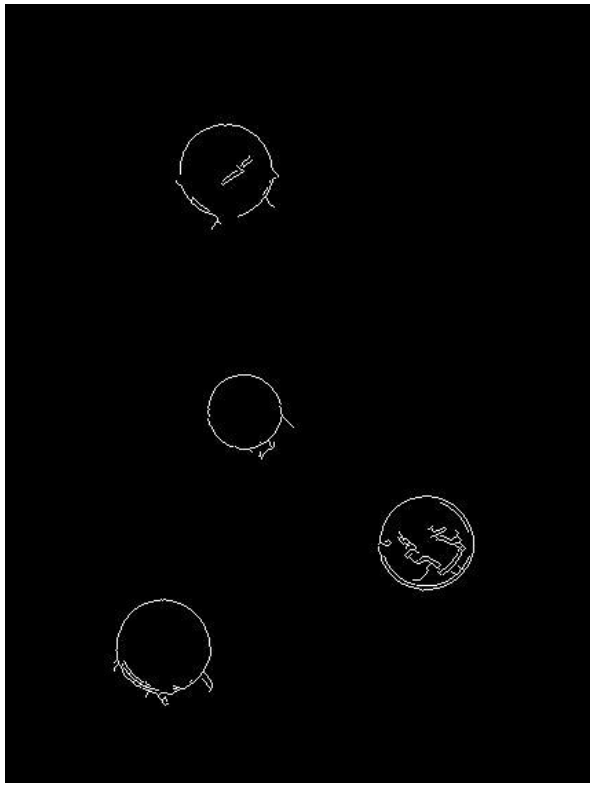
Example: detecting circles with Hough

Combined detections

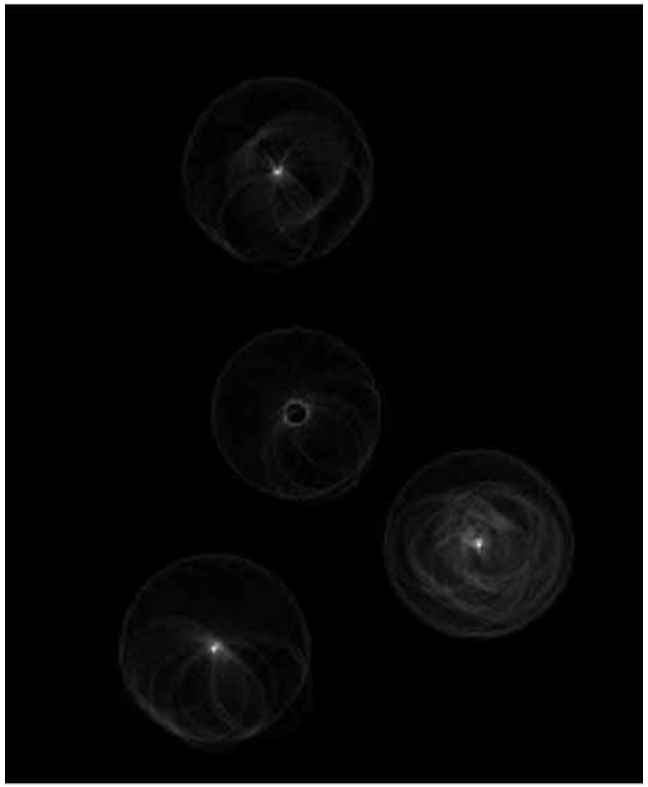
Original



Edges



Votes: Quarter



Coin finding sample images from: Vivek Kwatra

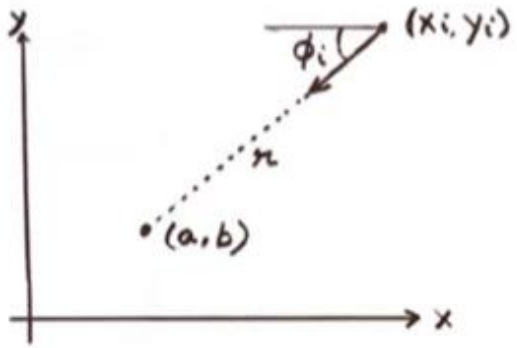
Hough Transform for other shapes can also be found

For a Circle Detection, it can be given as

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- Edge Location (x_i, y_i)
- Edge Direction ϕ_i

(a, b, r)



- $a = x - r\cos\phi$
- $b = y - r\sin\phi$

Load the image

Determine the image edges using any edge detector

Quantize the Parameter Space, P

Repeat the process for all pixels of image
If the pixel is an edge pixel, then calculate for all r

$$a = x - r\cos\phi$$
$$b = y - r\sin\phi$$

$$P(a,b,r) = P(a,b,r) + 1$$

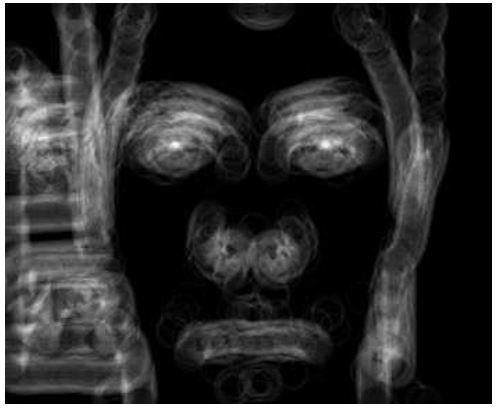
Show the Hough Space

Find the local maxima in parameter space

Draw the Circle using local maxima

Example: iris detection

Gradient+threshold



Hough space
(fixed radius)

Max detections

- Hemerson Pistori and Eduardo Rocha Costa http://rsbweb.nih.gov/ij/plugins/hough-circles.html

Example: iris detection

Figure 2. Original image

Figure 3. Distance image

Figure 4. Detected face region

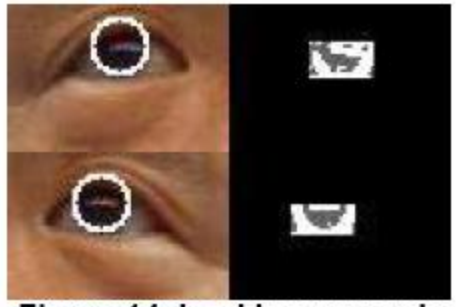


Figure 14. Looking upward

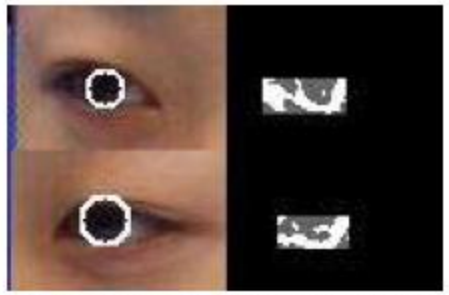


Figure 15. Looking sideways

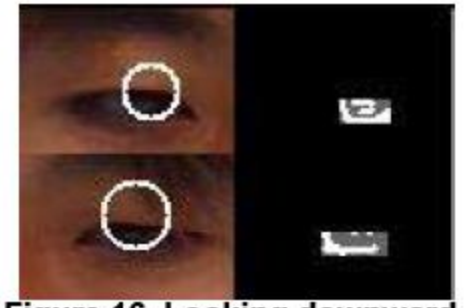


Figure 16. Looking downward

- An Iris Detection Method Using the Hough Transform and Its Evaluation for Facial and Eye Movement, by Hideki Kashima, Hitoshi Hongo, Kunihito Kato, Kazuhiko Yamamoto, ACCV 2002.

Hough transform: pros and cons

Pros

- All points are processed independently, so can cope with occlusion, gaps
- Some robustness to noise: noise points unlikely to contribute *consistently* to any single bin
- Can detect multiple instances of a model in a single pass

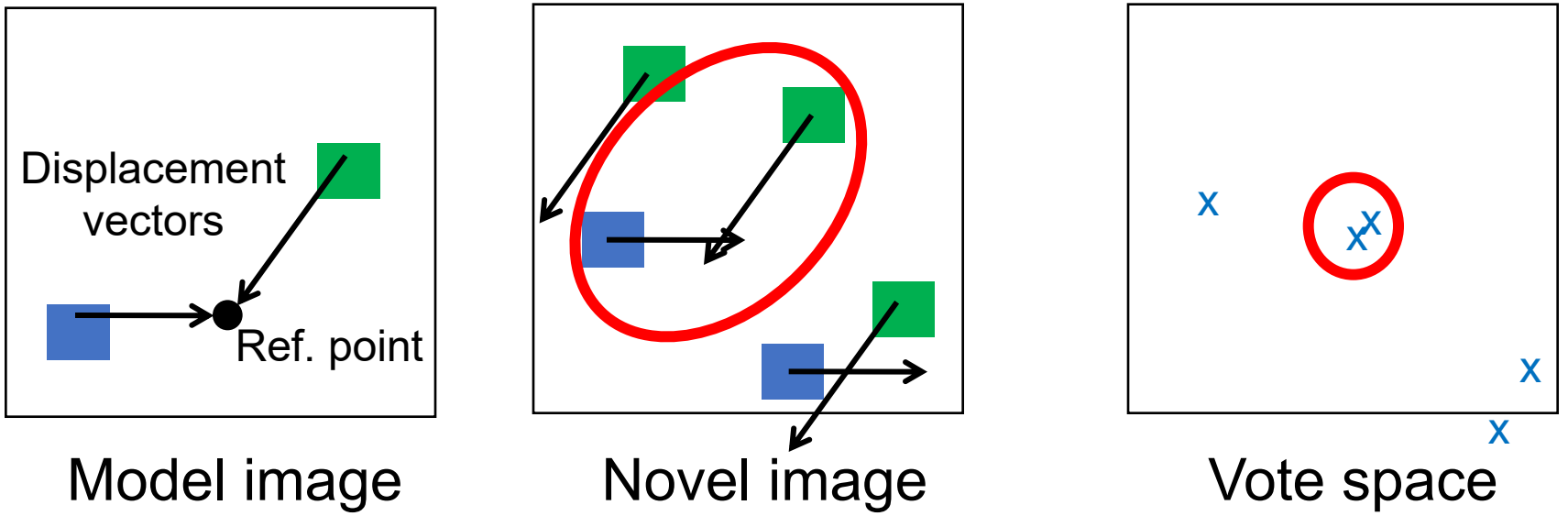
Cons

- Complexity of search time increases exponentially with the number of model parameters
- Non-target shapes can produce spurious peaks in parameter space
- Quantization: can be tricky to pick a good grid size

Generalized Hough Transform

- What if we want to detect arbitrary shapes?

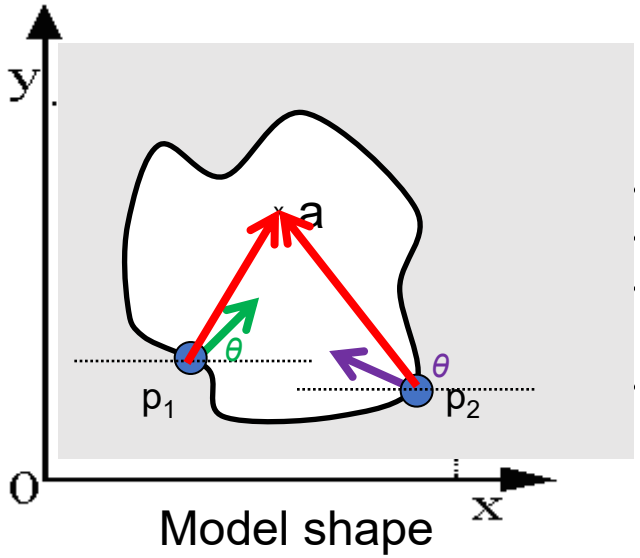
Intuition:



Now suppose those colors encode gradient directions…

Generalized Hough Transform

- Define a model shape by its boundary points and a reference point.



Model shape

θ	…
θ	…
⋮	

Offline procedure:

At each boundary point, compute displacement vector: r = a – p_i.

Store these vectors in a table indexed by gradient orientation θ.

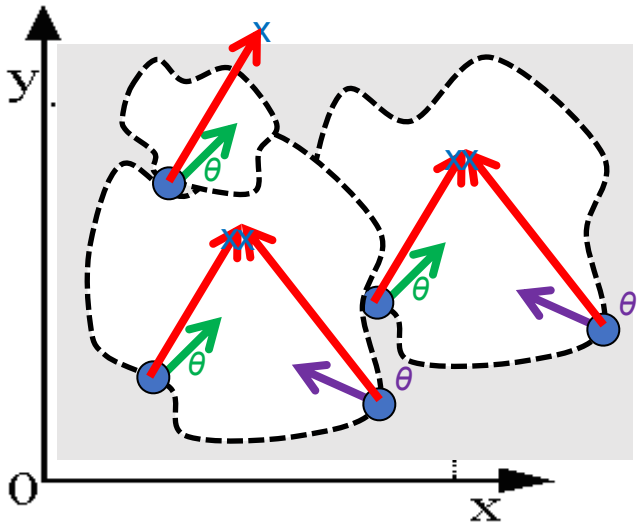
[Dana H. Ballard, Generalizing the Hough Transform to Detect Arbitrary Shapes, 1980]

Generalized Hough Transform

Detection procedure:

For each edge point:

- Use its gradient orientation θ to index into stored table
- Use retrieved r vectors to vote for reference point



θ	…
θ	…
⋮	

Assuming translation is the only transformation here, i.e., orientation and scale are fixed.

Summary

- **Fitting** problems require finding any supporting evidence for a model, even within clutter and missing features.
  - associate features with an explicit model

- **Voting** approaches, such as the **Hough transform**, make it possible to find likely model parameters without searching all combinations of features.
  - Hough transform approach for lines, circles, …, arbitrary shapes defined by a set of boundary points, recognition from patches.